

# Individual Developer Challenge: Build a Real-Time Collaborative Document Editor

## Duration

72 Hours

---

## Scenario

You have recently joined **SyncWrite**, a startup building cloud-based productivity software for businesses and educational institutions.

The company is preparing for an investor demonstration in two days. As one of the software engineers, you have been assigned to independently build a working prototype of a **real-time collaborative document editor** similar to Google Docs.

The objective is not to clone Google Docs feature-for-feature, but to demonstrate your ability to design and build a scalable, production-ready application that supports real-time collaboration between multiple users.

You are expected to make your own architectural decisions, select appropriate technologies, and deliver a complete working solution.

---

## Objective

Develop a web application that allows multiple authenticated users to create, edit, and collaborate on documents in real time.

Your implementation should demonstrate strong software engineering practices, clean architecture, maintainable code, and an excellent user experience.

---

## Functional Requirements

# Authentication

Implement secure user authentication.

Users should be able to:

- Register
- Login
- Logout

You may choose any authentication strategy.

---

## Dashboard

After authentication, users should see a dashboard displaying:

- Documents they own
  - Documents shared with them
  - Recently opened documents
  - Button to create a new document
- 

## Document Management

Users must be able to:

- Create documents
- Rename documents
- Delete documents
- Duplicate documents
- Open existing documents

Each document should include:

- Title
  - Owner
  - Date Created
  - Last Modified
-

# Rich Text Editing

Support at least the following:

- Headings
- Bold
- Italic
- Underline
- Lists
- Text Alignment
- Hyperlinks

You may build the editor yourself or integrate an open-source editor library.

---

## Main Requirement

### Real-Time Collaboration

When two or more users open the same document:

- Changes should appear instantly.
- No browser refresh should be required.
- All connected users should remain synchronized.
- The document should remain consistent.

Use an appropriate real-time communication technology such as WebSockets, Socket.IO, or Server-Sent Events.

---

### Presence Awareness

Users should be able to see who else is currently viewing the document.

Display:

- User names
- Online status

Bonus:

- Live cursor positions
  - Active editing location
- 

## Auto Save

Implement automatic saving.

Requirements:

- No Save button
  - Automatically persist changes
  - Prevent data loss after browser refresh
- 

## Version History

Maintain previous versions of documents.

Users should be able to:

- View previous revisions
  - Restore an earlier version
  - See when revisions were made
  - See who created each revision
- 

## Comments

Users should be able to:

- Add comments
  - Reply to comments
  - Resolve comments
  - Delete their own comments
-

# Sharing & Permissions

Support document sharing.

Permission levels:

- Viewer
- Commenter
- Editor

Unauthorized users must not be able to modify document content.

---

# Technical Expectations

Your solution should demonstrate:

- Clean architecture
  - Proper folder structure
  - Modular components
  - Reusable code
  - Validation
  - Error handling
  - Authorization
  - Responsive design
  - Efficient database design
- 

# Bonus Features

Implement any additional features that improve the application.

Examples include:

- Live cursor tracking
- Typing indicators
- Keyboard shortcuts
- Document search

- Dark mode
  - Export to PDF or Markdown
  - Import Markdown
  - Offline editing with synchronization
  - User avatars
  - Notifications
- 

## Constraints

- You must complete the project independently.
  - You may use open-source libraries, but you must understand how they work and be prepared to explain your implementation.
  - AI tools may be used only as references. Your code should reflect your own understanding.
  - Your application should be deployable and runnable using the provided instructions.
- 

## Deliverables

Submit the following:

- Source code
  - README with setup instructions
  - Database schema
  - API documentation (if applicable)
  - Short demonstration video (5–10 minutes)
- 

## Final Note

This challenge is intentionally designed to simulate a real software engineering task rather than a tutorial project.

Your evaluator will assess not only whether the application works, but also how you approached the problem, structured your code, handled edge cases, and justified your

technical decisions.

Focus on building a solution that is clean, scalable, maintainable, and demonstrates your ability to solve complex engineering problems independently within a limited timeframe.